COMPUTER VISION · SPORTS ANALYTICS

FOOTBALL PLAYER DETECTION

From broadcast frames to team & possession analytics with YOLOv8

Shubham Tanna

Python · Ultralytics YOLOv8 · OpenCV · scikit-learn

REAL-TIME SPORTS ANALYTICS

01

THE CHALLENGE

Broadcast footage is hard for computers: 20+ players who look alike, a ball only a few pixels wide, changing light and constant camera movement.

02

OBJECTIVE

Build a model that finds and labels every **player, goalkeeper, referee and the ball** in each frame, then turn those detections into match information.

03

WHY IT MATTERS

Detection is the first step for tactical analysis, player tracking, automated highlights and referee assistance, all of which today rely on costly manual tagging or sensors.

SCOPE

372 Bundesliga broadcast frames · 4 classes · YOLOv8-nano fine-tuned on CPU · test-set evaluation · team split by jersey colour · possession estimate

WHAT THE RESEARCH SAYS

Literature review of three recent papers, which shaped the design choices below

WANG & LI, 2022

LIGHTWEIGHT CNN-FPN

0.915 AP on the ISSIA-CNR benchmark, ahead of Faster R-CNN, using a small network suited to portable devices.

Small and occluded players remain hard.

MERZAH ET AL., 2024

OPENCV TRACKING

Low-cost CSRT tracking turns positions into speed and distance, with no GPS hardware needed.

Each player must be selected by hand, so it doesn't scale.

ZHAO, 2024

UPGRADED YOLOV5

mAP@0.5 rose from 0.81 to 0.89 with GFLOPs nearly halved, and it detects the ball as well as players.

Stricter mAP@0.5:0.95 stayed flat.

My takeaway → use a compact single-stage YOLO detector, keep the ball as its own class and report it separately, and build analytics on top of the detections rather than stopping at boxes.

THE DATA

372

BROADCAST FRAMES · 1280×1280

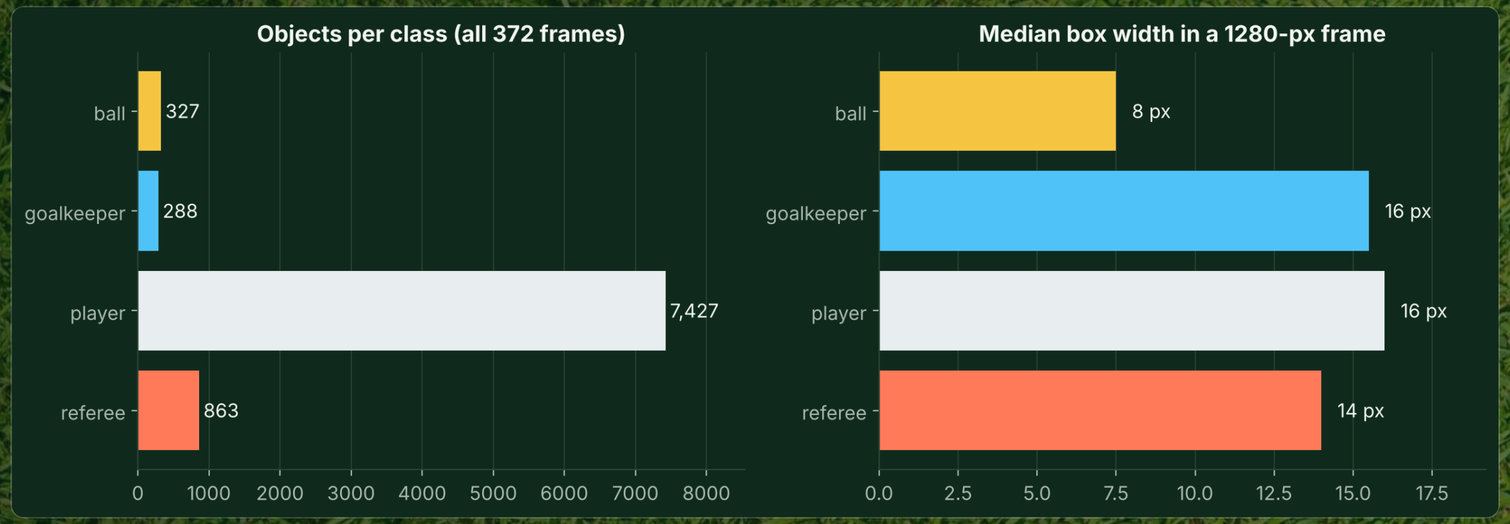
8,905

LABELLED OBJECTS · 4 CLASSES

298 / 49 / 25

TRAIN / VALIDATION / TEST FRAMES

Source: Roboflow Universe public football-players-detection dataset (DFL Bundesliga clips), CC BY 4.0



Key challenge: players are 23× more common than the ball, and the median ball is just 8 px wide, under 1% of the frame. The ball is expected to be the weakest class.

THE PIPELINE

1
PREPARE
Roboflow export in YOLO format. Frames resized to 640 px for training. Mosaic, colour and flip augmentation.

2
TRAIN
YOLOv8-nano (3M parameters), COCO-pretrained, fine-tuned for 43 epochs on a 2-core CPU. No GPU.

3
EVALUATE
Held-out test set: precision, recall, mAP per class, confusion matrix, inference speed.

4
ANALYSE
Jersey-colour k-means splits players into teams. Nearest player to the ball gives an estimate of possession.

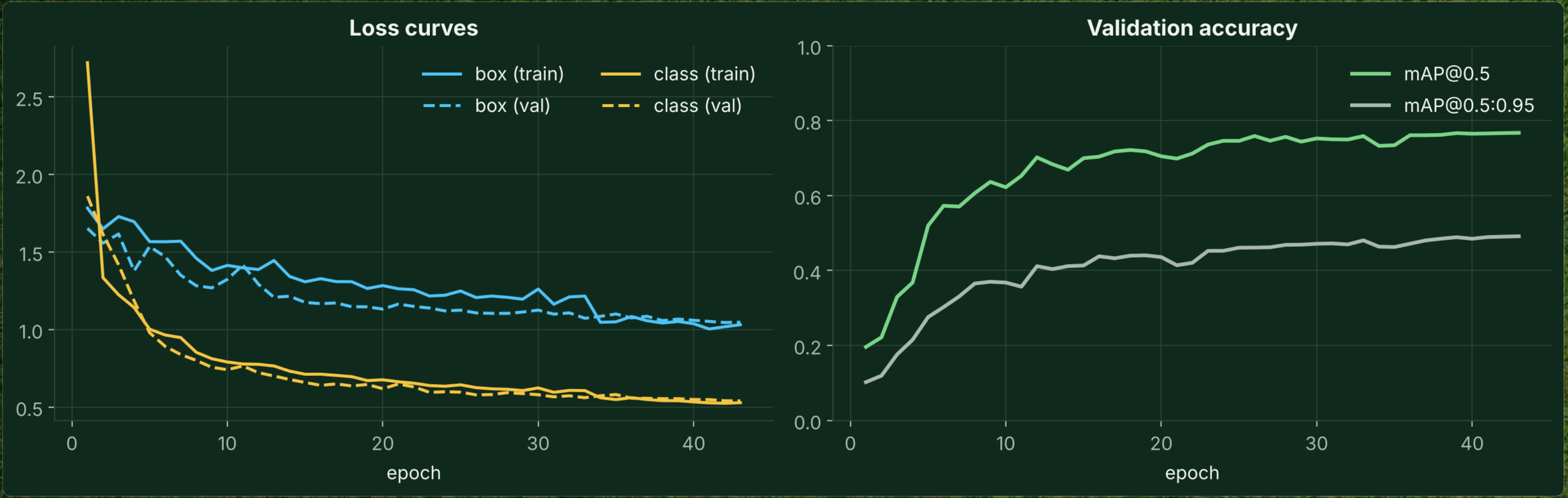
VERSION 1

Trained on Roboflow's hosted platform (Roboflow 3.0 Object Detection). Quick to build, but a black box.

VERSION 2 · THIS PROJECT

Rebuilt end to end in Python with Ultralytics: every step can be read, rerun and extended, and the analytics layer is added.

TRAINING



Transfer learning: starting from COCO weights, validation mAP@0.5 passed 0.5 within 5 epochs.

No overfitting: validation losses track training losses closely all the way through.

Budget-bound: 43 epochs in 1.75 h on a 2-core CPU. Accuracy was still rising slowly, so more training would help.

RESULTS ON THE TEST SET

0.83
MAP@0.5 · ALL CLASSES

0.99
MAP@0.5 · PLAYERS

0.43
MAP@0.5 · BALL

4.2
FRAMES / SEC ON CPU

CLASS	OBJECTS	PRECISION	RECALL	MAP@.5	MAP@.5:.95
ball	24	0.95	0.38	0.43	0.18
goalkeeper	19	0.94	0.78	0.93	0.72
player	499	0.98	0.97	0.99	0.79
referee	56	0.93	0.88	0.95	0.63
all	**598**	**0.95**	**0.75**	**0.83**	**0.58**

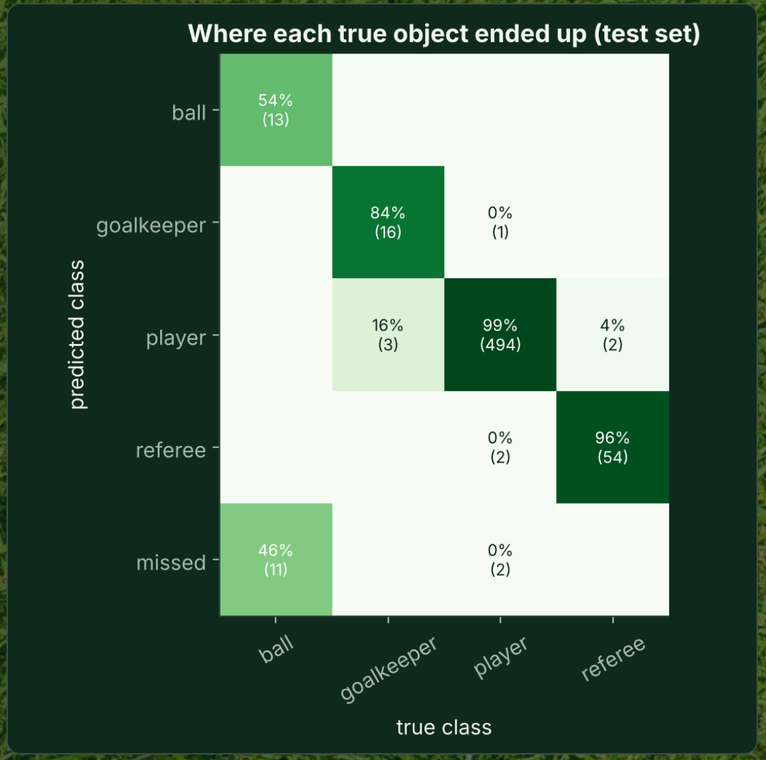
READING THE NUMBERS

- Players, goalkeepers and referees are found almost perfectly (0.93–0.99).
- The ball is the weak spot: **95% precision** but **38% recall**. When the model calls a ball it is almost always right, but it misses most of them.
- Choosing a 960 px inference size on validation data lifted ball mAP from 0.28 to 0.43 and overall from 0.79 to 0.83, versus 640 px.
- Overall this matches v1 (0.82) and beats it on 3 of 4 classes.

WHAT THE MODEL SEES

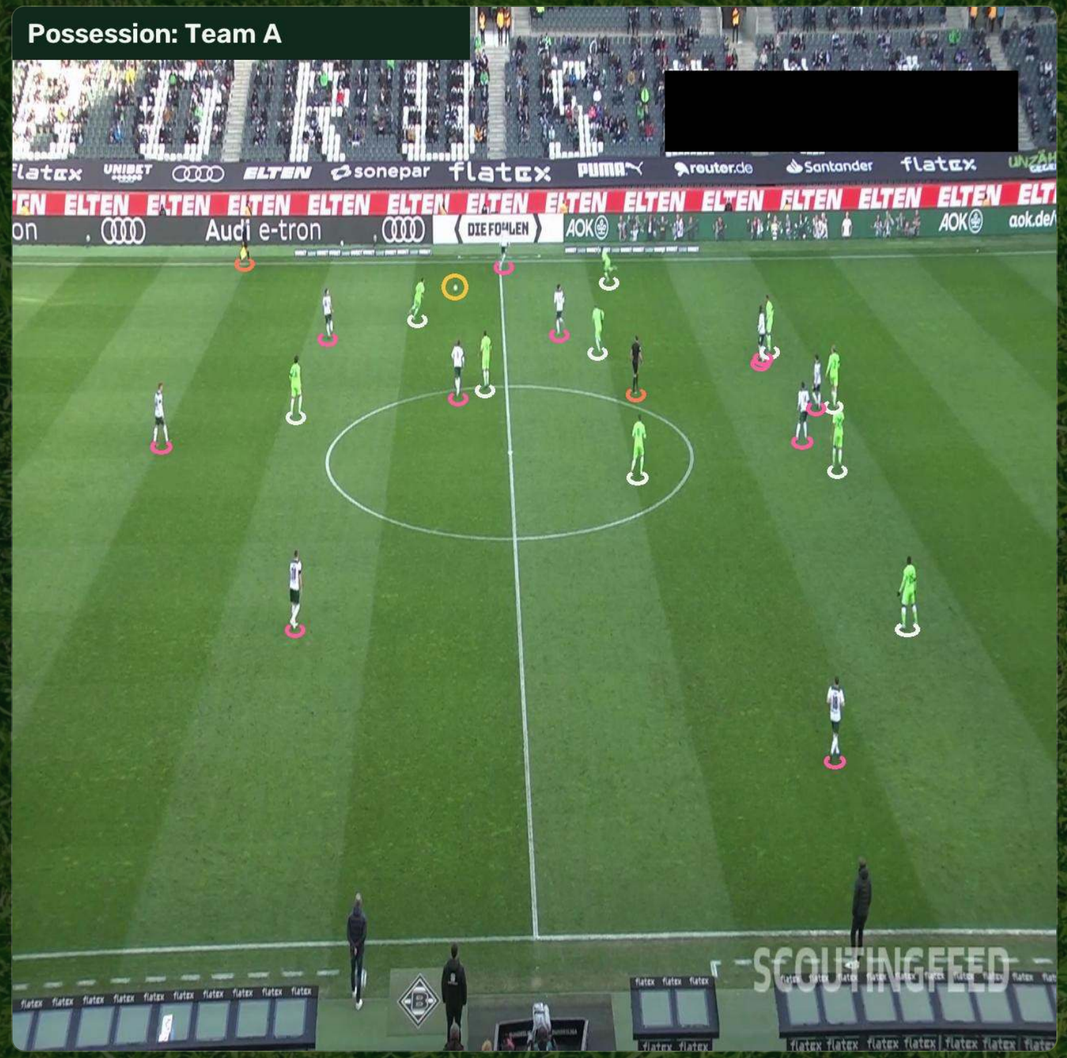


Top: hand-labelled ground truth. Bottom: model predictions on unseen test frames.



99% of players and 96% of referees are classified correctly. The main confusion is 3 goalkeepers labelled as players, which is understandable since they look alike. **46% of balls are missed entirely.**

FROM BOXES TO MATCH INSIGHT

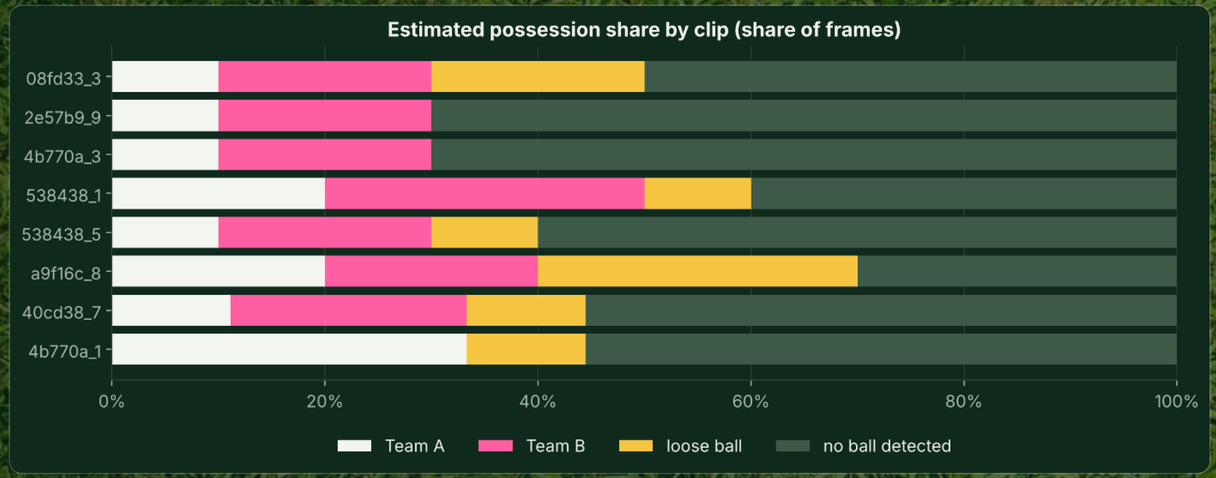


TEAM IDENTIFICATION

The colour at the centre of each player's shirt is clustered into 2 teams per clip with k-means. No kit labels are needed.

POSSESSION ESTIMATE

The team of the player whose feet are closest to the ball, within 1.5 body-heights, has possession in that frame.



Across all 372 frames: Team A 16%, Team B 24%, loose ball 8%. In 53% of frames no ball is detected, so better ball recall is the biggest lever for this analysis.

CONCLUSION & NEXT STEPS

KEY FINDINGS

- A 3M-parameter model trained on a laptop-class CPU detects people on the pitch with **0.93–0.99 mAP@0.5**.
- Tuning the inference resolution was a cheap win: **+0.04 overall, +0.14 on the ball**, with no retraining.
- The ball limits everything downstream. Possession can only be estimated in frames where the ball is found.
- Test frames come from the same clips as the training frames, so these scores are an upper bound.

NEXT STEPS

- **Ball:** train at 1280 px on a GPU or use tiled inference (SAHI) to recover tiny objects.
- **Stricter test:** hold out entire matches, not just frames, to measure true generalisation.
- **Tracking:** add ByteTrack and a pitch homography to get distance covered, heatmaps and possession in seconds.

Code, notebook & data credits: see project README

Shubham Tanna